Programming for Problem Solving Using Python

Unit-1

Session 9

List Comprehensions

Iflah Aijaz

School of Engineering & Technology (SOET)
K. R. Managalam University
Program: BTech (CSE)/BTech Full Stack Xebia/BTech UX UI/
B.Sc. DS/B.Sc. Cyber/MC

Session-9

List Comprehensions

List Comprehensions

- A **list comprehension** is a concise way to create lists.
- Combines **loop** + **expression** in one line.
- Faster & cleaner than traditional loops.
- Syntax:

```
[expression for item in iterable]
```

Why List Comprehensions?

- Shorter and more readable code.
- Often faster than for loops.
- Easy to transform and filter data.
- Ideal for mathematical, string, and conditional operations.

Basic Syntax

```python
nums = [1, 2, 3, 4]
squares = [n**2 for n in nums]
print(squares)  # [1, 4, 9, 16]
```

```
[1, 4, 9, 16]
```

Simple Arithmetic Expressions

- **Expression**: n * 2
- **Iterable**: numbers
- **Result**: Transformed list.

```
numbers = [1, 2, 3, 4, 5]
doubled = [n * 2 for n in numbers]
print(doubled)  # [2, 4, 6, 8, 10]
```

```
[2, 4, 6, 8, 10]
```

String Manipulation

```python
words = ["python", "rocks"]
upper_words = [w.upper() for w in words]
print(upper_words)  # ['PYTHON', 'ROCKS']
```

```
['PYTHON', 'ROCKS']
```

Conditional Expressions

```
[expression for item in iterable if condition]
```

```
nums = [1, 2, 3, 4, 5]
evens = [n for n in nums if n % 2 == 0]
print(evens)  # [2, 4]
```

```
[2, 4]
```

Conditional with Else (Ternary)

```
nums = [1, 2, 3, 4]
labels = ["even" if n % 2 == 0 else "odd" for n in nums]
print(labels)  # ['odd', 'even', 'odd', 'even']
```

```
['odd', 'even', 'odd', 'even']
```

Using Functions in Expressions

```
def cube(n): return n**3
nums = [1, 2, 3]
cubes = [cube(x) for x in nums]
print(cubes)  # [1, 8, 27]
```

```
[1, 8, 27]
```

Nested Loops in Comprehensions

```
pairs = [(x, y) for x in [1, 2] for y in ['a', 'b']]
print(pairs)
```

```
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
```

Filtering with Multiple Conditions

```
nums = range(20)
filtered = [n for n in nums if n % 2 == 0 if n > 10]
print(filtered)  # [12, 14, 16, 18]
```

```
[12, 14, 16, 18]
```

Complex Expressions

```
nums = [1, 2, 3]
complex_expr = [(n, n**2, n**3) for n in nums]
print(complex_expr)  # [(1, 1, 1), (2, 4, 8), (3, 9, 27)]
```

```
[(1, 1, 1), (2, 4, 8), (3, 9, 27)]
```

List Comprehension with range()

```
multiples = [n*5 for n in range(1, 6)]
print(multiples)  # [5, 10, 15, 20, 25]
```

```
[5, 10, 15, 20, 25]
```

Flattening Lists

```
matrix = [[1, 2], [3, 4]]
flat = [num for row in matrix for num in row]
print(flat)  # [1, 2, 3, 4]
```

```
[1, 2, 3, 4]
```

Using Built-in Functions

```
words = ["apple", "banana"]
lengths = [len(w) for w in words]
print(lengths)  # [5, 6]
```

```
[5, 6]
```

Removing Duplicates

```
nums = [1, 2, 2, 3]
unique = list({n for n in nums})
print(unique)  # [1, 2, 3]
```

```
[1, 2, 3]
```

Using zip() with Comprehensions

```
names = ["Alice", "Bob"]
scores = [85, 90]
combined = [f"{n}: {s}" for n, s in zip(names, scores)]
print(combined)  # ['Alice: 85', 'Bob: 90']
```

```
['Alice: 85', 'Bob: 90']
```

Performance Note

- List comprehensions are generally **faster** than loops.
- But avoid making them too complex — readability matters.
- Good for transformation, filtering, and mapping tasks.

Thank You!

Any Questions?